Taller 2do corte – Desarrollo de FrontEnd

César Fabricio Salas Ricaurte – T00065846 – Ingeniería de Sistemas
Curso: Desarrollo de FrontEnd
Maestro: Luis Giovanny Carreño Ortiz

Punto 1:

React se comporta como una biblioteca de JavaScript en formato de código abierto, teniendo como equipo de creadores de la compañía Meta (que son los mismos dueños de Facebook) en conjunto con una gran comunidad de desarrolladores independientes lanzado de manera oficial en el año del 2013.
Se ha consolidado como una de las tecnologías FrontEnd más usadas en la actualidad ya que permite crear y construir interfaces y experiencias de usuario dinámicas y escalables, bastante enfocado en las aplicaciones web móviles.

A menudo puede llegar a confundirse los conceptos de “Framework” y “Librería” cuando hablamos de React pero para dejarlo en claro, básicamente:

A enfoque:

- Librería (React): Se centra solo en la vista y en cómo mostrar todos los componentes en la pantalla.
- Framework: Ofrece una solución completa y compacta “Todo en uno” con reglas estrictas y con una estructura fija.

En control:

- Con React, tu puedes decidir que herramientas extra usar para el enrutamiento o la gestión de datos.
- Con un Framework, es la estructura la que dicta cómo es que se debe organizar y ejecutar tu código.

Y con todo esto ya un poco comentado, para crear una aplicación web completa con React, su sitio oficial y los que más experiencia tienen recomiendan casi siempre apoyarse en meta-frameworks basados en él, como Next.js.

Algo curioso de esta librería, es que te da la libertad de estado en específico, es decir, nosotros mismos describimos como queremos que se vea la interfaz de usuario para un estado específico ya que el mismo React se encarga de hacer los cambios necesarios en el DOM (o sea, la estructura de la página) por ti.

Punto 2:

La dificultad principal de construir interfaces con JavaScript a mano por así decirlo manipulando el DOM, era mantener la interfaz sincronizada con los datos de la aplicación; cada vez que los datos cambiaban, el que estaba programando tenía que actualizar manualmente los elementos del DOM, como textos, botones, listas o formularios, todo esto además hacia que el código fuera bien complicado y que hacia que fuera difícil de darle mantenimiento.

Además de esto podemos abarcar un poco 2 tipos de enfoque en la construcción de aplicaciones con estas tecnologías, es decir, de manera imperativa o declarativa, respectivamente:

- Imperativa: Consiste en indicar paso por paso como se tiene que realizar una tarea. En JavaScript, se tiene que manipular directamente el DOM para modificar la interfaz.
- Declarativo: Consiste en describir como se debe ver la interfaz según los datos, actuales, dejando que la propia tecnología se encargue de actualizarlo.

Una de las cosas mas interesantes de React, es que permite que la interfaz se actualice automáticamente cuando cambia el estado de un componente, todo esto cuando los datos cambian, React vuelve a evaluar como debe verse la interfaz y actualiza solito el DOM para reflejar esos cambios, así nadie tiene que manipular cada elemento de la pantalla.

Todo esto conlleva a reutilizar componentes que permiten dividir una o varias interfaces en pequeñas partes independiente, con que cada una se le asigne una función específica, teniendo como ventaja la reutilización, mantenimiento, organización y sincronización de la estructura de la aplicación que se esté trabajando.

Punto 3:

Elementos desarrollados:

Componentes

Son partes independientes y reutilizables de una interfaz de usuario. Cada componente puede representar un elemento como un botón, una tarjeta, un menú o incluso una página completa.

Estos permiten dividir un app en partes pequeñas y organizadas, se pueden reutilizar en varios lugares, además pueden recibir información, pueden también tener su propio estado y responder a eventos, en React normalmente se crean mediante funciones de JavaScript que devuelven JSX, por decir un ejemplo:

```
function Saludo() {
  return <h1>¡Hola, mundo!</h1>;
}

export default Saludo;
```

Aquí arriba se crea un componente llamado -Saludo- que muestra un encabezado con el mensaje “Hola, mundo!”. Se puede reutilizar en otra parte de la app escribiendo en ella **<Saludo/>**

JSX

Es una extensión de la sintaxis de JavaScript que permite escribir estructuras similares a HTML dentro del código JavaScript. React lo utiliza habitualmente para describir cómo debe verse la interfaz.

JSX combina JavaScript con una sintaxis similar a HTML; los elementos deben cerrarse correctamente; los componentes se escriben con la primera letra en mayúscula; para insertar expresiones de JavaScript se utilizan llaves { }; algunos atributos tienen nombres diferentes a los de HTML, como className en lugar de class.

```
function Presentacion() {
  const nombre = "Carlos";

  return (
    <div>
      <h1>Bienvenido</h1>
      <p>Hola, {nombre}</p>
    </div>
  );
}
```

Este código muestra un titulo y párrafo, la expresión que se ve {nombre} inserta el valor de la variable dentro del contenido de JSX

Props

son datos que un componente recibe desde el componente que lo utiliza. Sirven para personalizar su contenido y comportamiento sin tener que crear un componente diferente para cada caso.

Permiten pasar información entre componentes, normalmente del padre al hijo; Pueden contener textos, números, valores booleanos, funciones y otros datos; Son de solo lectura para el componente que las recibe; Facilitan la reutilización de componentes con información diferente.

```
function Saludo(props) {
  return <h1>Hola, {props.nombre}</h1>;
}

function App() {
  return (
    <div>
      <Saludo nombre="Ana" />
      <Saludo nombre="Pedro" />
    </div>
  );
}
```

Este componente Saludo recibe la prop nombre. Al utilizarlo con valores diferentes, muestra «Hola, Ana» y «Hola, Pedro». De esta manera, un mismo componente puede reutilizarse con distintos datos.

Estado (useState)

Estado es la información que un componente necesita recordar mientras la aplicación funciona y que puede cambiar con el tiempo. React proporciona el Hook useState para declarar y actualizar ese estado.

Permite almacenar datos que pueden cambiar, como un contador, un nombre o la selección de un usuario; useState devuelve el valor actual y una función para actualizarlo; Cuando el estado cambia mediante su función actualizadora, React vuelve a renderizar el componente para reflejar el nuevo valor; No se debe modificar directamente la variable del estado; Un componente puede tener varios estados.

```
import { useState } from "react";

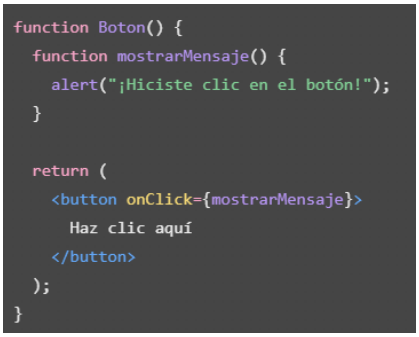
function Contador() {
  const [contador, setContador] = useState(0);

  return (
    <div>
      <p>Contador: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>
        Aumentar
      </button>
    </div>
  );
}
```

Aquí useState(0) establece el valor inicial del contador en cero, contador contiene el valor actual, setContador permite actualizar ese valor, Al hacer clic en el botón, el contador aumenta en uno y React actualiza el número mostrado en pantalla (Que esto básicamente es el que se muestra en la página principal cuando creamos nuestro primer proyecto de React).

Eventos (onClick, onChange)

Estos son los mecanismos que permiten que una aplicación responda a las acciones del usuario, como hacer clic en un botón, escribir en un campo de texto o seleccionar una opción.
Permiten crear interfaces interactivas, onClick ejecuta una función cuando el usuario hace clic en un elemento; onChange permite responder a cambios en controles de formulario, como campos de texto. Se utilizan para actualizar el estado, validar datos o ejecutar otras acciones.



```
function Boton() {
  function mostrarMensaje() {
    alert("¡Hiciste clic en el botón!");
  }

  return (
    <button onClick={mostrarMensaje}>
      Haz clic aquí
    </button>
  );
}
```

En palabras simples, cuando el usuario hace clic en el botón, React ejecuta la función -mostrarMensaje-, que muestra una ventana emergente con un mensaje.

Esto es un evento onClick



```
import { useState } from "react";

function CampoNombre() {
  const [nombre, setNombre] = useState("");

  return (
    <div>
      <input
        type="text"
        value={nombre}
        onChange={(evento) => setNombre(evento.target.value)}
        placeholder="Escribe tu nombre"
      />

      <p>Tu nombre es: {nombre}</p>
    </div>
  );
}
```

El campo de texto guarda lo que escribe el usuario en el estado nombre. La función “onChange” actualiza ese estado y React muestra el texto actualizado en el párrafo.

Este ahora es un evento onClick.

Punto 4

- ¿Por qué tantas empresas eligen React para sus productos web?
- ¿Qué aporta React al **ecosistema** de desarrollo (herramientas que lo rodean: empaquetadores, frameworks basados en React, etc.)?
- ¿Qué papel juegan su **comunidad** y su mantenimiento (quién lo mantiene hoy)?
- ¿Por qué es una habilidad **relevante** para un desarrollador front-end hoy en día?

La empresas últimamente se están inclinando por React porque les permite desarrollar interfaces dinámicas, organizadas y fáciles manejar; habíamos hablado anteriormente de que se podían reutlizar los componentes de React que nos permiten además crear elementos como botones, fomularios, etc.
Tambien que nos deja describir la interfaz según los datos actuales y actualizar lo necesario cuando estos cambian, y que también nos puede permitirnos integrar en proyectos existentes o utilizarse para construir aplicaciones y herramientas completas en funcionalidades y complejas.
Por ejemplo, una empresa de comercio electrónico puede utilizar componente compartidos para mostrar productos, gestionar un carrito de comparas y construir formularios de pago, sin tener que desarrollar cada elemento desde cero, tenemos que aclarar que React no garantiza al 100% por sí solo una aplicación más rápida o segura. Los resultados también dependen de la arquitectura, decisiones técnicas y la calidad de la implementación.

Al hablar de todo esto, React cuanta con un amplia gama de ecosistema con el que se puede trabajar con él, ya que no trabaja de manera aislada, necesita de otras tecnologías para funcionar de manera optima:

- Vite o Webpack: nos ayudan a preparar el código para le navegador, administrar los recursos y facilitar el desarrollo local

- Next.js: es un Framework basado casi completamente en React, que nos permite construir aplicaciones con funciones adicionales como renderizado en el servidor, nos genera paginas estáticas, enrutamientos y opciones para desarrollar apps de manera muy completa.

- React Router: ayuda a organizar las rutas de una app. Otras bibliotecas permiten gestionar datos remotos, formularios y estados compartidos.

Vitest y React testing library: permiten verificar el funcionamiento de los componentes y detectar errores antes de publicar una aplicación.

La comunidad es uno de los factores que contribuyen a la continuidad y popularidad de React. Al ser un proyecto de código abierto, los desarrolladores pueden estudiar su código, reportar problemas, proponer mejoras, crear bibliotecas y compartir conocimientos.
Desde febrero de 2026, React y React Native forman parte de la React Foundation, que proporciona una estructura independiente para apoyar el proyecto y su ecosistema. Entre sus miembros fundadores se encuentran Meta, Microsoft, Amazon, Vercel, Expo, Huawei, Callstack y Software Mansion.

Debemos prestar atención en aprender React, es importante porque es una tecnología muy utilizada en el desarrollo web. Permite ampliar las oportunidades laborales, crear aplicaciones reales, integrarse con otras tecnologías y desarrollar proyectos complejos mediante componentes, estado y eventos. Además, su uso frecuente en aplicaciones web demuestra su relevancia en el ámbito profesional.

Punto 5

Para comentar 2 ejemplos de empresas reconocidas que utilicen React, serían:

Meta (Facebook e Instagram)

Meta utiliza React en productos web como Facebook e Instagram. En su artículo técnico sobre React Compiler, el equipo de React explica que la tecnología se implementó en Instagram y posteriormente en otras aplicaciones web importantes, entre ellas Facebook y Threads.

Por qué es un caso relevante?

- Permite desarrollar interfaces con publicaciones, comentarios, notificaciones y contenido que cambia constantemente.
- Demuestra que React puede utilizarse en aplicaciones de gran escala.
- Meta puede aplicar mejoras al rendimiento y a la experiencia de desarrollo en productos con numerosos componentes.

- React Labs: What We've Been Working On – February 2024. (2026). Retrieved 16 October 2026, from https://react.dev/blog/2024/02/15/react-labs-what-we-have-been-working-on-february-2024?utm_source=

Airbnb

Airbnb cuenta con una trayectoria documentada de desarrollo con React. Su equipo de ingeniería ha publicado artículos técnicos sobre componentes React, pruebas de interfaces y herramientas internas para desarrollar aplicaciones interactivas. Entre los ejemplos se encuentran sus esfuerzos por migrar miles de archivos de pruebas de componentes a React Testing Library.

Por qué es un caso relevante?

- Una plataforma de reservas necesita interfaces interactivas para buscar alojamientos, modificar filtros y gestionar información de viajes.
- Los componentes reutilizables ayudan a mantener una experiencia visual coherente en diferentes secciones.
- Su experiencia con pruebas de componentes demuestra cómo React puede integrarse en procesos profesionales de mantenimiento y calidad del software.

- How Airbnb Uses React. (2026). Retrieved 16 October 2026, from https://www.engineering.fyi/company/airbnb/react?utm_source

Referencias:

- *Passing props to a component – react*. (n.d.). https://react.dev/learn/passing-props-to-a-component
- *Vite*. (n.d.). Vitejs. https://vite.dev/
- Peiro, E. (2026, April 24). ¿Qué es React? La librería de JavaScript para interfaces de usuario dinámicas. *Aprender21*. https://www.aprender21.com/blog/que-es-react-javascript-libreria-frontend
- *Concepts | webpack*. (n.d.). Webpack. https://webpack.js.org/concepts/
- Durán, M. Á. (2022, October 12). ¿Es React una biblioteca o un framework? *midudev*. https://midu.dev/react-biblioteca-o-framework/
- *Next.js docs*. (n.d.). https://nextjs.org/docs
- Vladimir, A. F. H. O. a. P. J. S. P. R. M. a. V. C. (n.d.). *Vitest*. Next Generation Testing Framework. https://vitest.dev/
- *React Testing Library | Testing Library*. (2024, June 3). https://testing-library.com/docs/react-testing-library/intro/
- *REACT Foundation - supporting the REACT ecosystem*. (n.d.). React Foundation. https://www.react.foundation/about
- *Quick start – react*. (n.d.). https://react.dev/learn
- *State of React 2025: usage*. (n.d.). https://2025.stateofreact.com/so-SO/usage/
- Saavedra, J. A. (2023, July 17). *Qué es React y para qué sirve*. Ebac. https://ebac.mx/blog/que-es-react